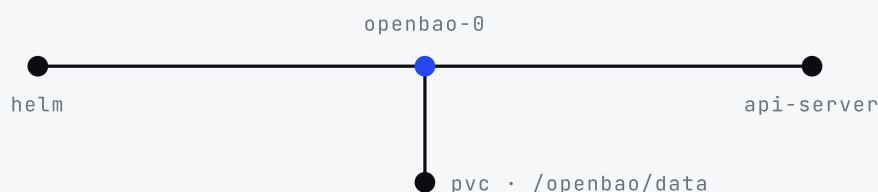


OPENBAO ON KUBERNETES

A secrets store inside the cluster: Helm, Raft, init and unseal, Kubernetes auth – and what is still missing afterwards.



OpenBao on Kubernetes

A secrets store inside the cluster: Helm, Raft, init and unseal, Kubernetes auth – and what is still missing afterwards

Thomas Zachmann

17 September 2026

Contents

What this is about

- Why by hand, and why without AI
- A word about the values in this guide
- License and liability
- Who this is for
- The building blocks
- The repository
- The cluster
- On kind instead of RKE2

Part I – Helm

- The chart
- Why one pod and not three
- The values
- Installing
- What `helm upgrade` does *not* do later

Part II – init and unseal

- Why this is not automated
- init
- unseal
- What happens after every restart
- The first login

Part III – Kubernetes auth

- How it works
- Enable and configure
- A role for one consumer
- Checking without ESO

Part IV – Access from outside

- The path
- The `bao` CLI from outside
- The UI

Part V – The current state, honestly

Part VI – Hardening: what comes next

1. Remove the token file from the pod
2. Audit device
3. Admin policy and userpass
4. Prove the replacement access – do not skip
5. Revoke the root token
6. A first snapshot

Part VII – What can go wrong

Pod stays 0/1 Running

Log full of service_registration ... 403

helm upgrade changes nothing

403 permission denied on everything

Kubernetes login fails

OpenBao stops accepting requests, log mentions audit

generate-root answers 405 unsupported operation

After node maintenance: sealed

Appendix A – All commands

Appendix B – Glossary

Appendix C – The reverse proxy

Why DuckDNS, and why DNS-01

1. DuckDNS

2. The proxy VM

3. The certificate

4. The proxy host

Check

What is missing here, honestly

About the author

What this is about

Kubernetes ships its own object for secrets, the Secret. It stores passwords and keys, and that is all: it does not log who reads them, it does not issue credentials with an expiry, and it does not tell apart which workload is asking. A cluster therefore needs a place for secrets that can do those three things. This guide builds exactly that: **OpenBao inside the cluster**, as a StatefulSet with Raft storage, installed via Helm, initialised and unsealed by hand, with Kubernetes auth as the login method for workloads.

It is the first part of a series. The second, *Dynamic Database Credentials with OpenBao*, builds on precisely this setup and lets an application draw short-lived PostgreSQL access from it. The third describes the same OpenBao on a virtual machine – with Ansible, OpenTofu and a rehearsed disaster recovery. Many decisions in this guide are taken from there, and wherever the cluster setup falls *behind* the VM setup, that is said explicitly.

Everything here was actually carried out on an RKE2 cluster in a lab. The state at the end is described honestly – including what has not been done yet.

Why by hand, and why without AI

In a real setup a Helm release is not deployed by hand but via GitOps, and the OpenBao configuration not via CLI but via OpenTofu. Even so, this guide walks every step once by hand – and deliberately without AI assistance as a shortcut. Whoever lets a language model write the values gets a running OpenBao and no idea why `tlsDisable` is set, what distinguishes Raft from `file`, or why the pod is sealed after every restart.

The yardstick: whoever has worked through this guide can draw, on a blank sheet, how a pod in the cluster gets to a secret in OpenBao – through which objects, with which token, verified by whom. Whoever can do that can also hook the Helm release into Flux or Argo and move the configuration to OpenTofu.

A word about the values in this guide

The cluster is a development environment: no access from outside, no real data. Hostnames and internal addresses are replaced by placeholders (`bao.example.internal`). Unseal keys and the root token appear nowhere – they are not in the original terminal log either, because `init` deliberately never runs inside an automation or a chat window.

License and liability

This guide is licensed under CC BY 4.0: it may be copied, shared and adapted, commercially too, as long as the author is credited. It is provided as is, without warranty. Everything in it was carried out in a development environment; whoever reproduces it elsewhere does so at their own risk.

Who this is for

Readers who know Kubernetes basics (StatefulSet, PVC, Ingress, ServiceAccount, Helm) and know what a secrets manager is. OpenBao is a fork of HashiCorp Vault; everything here applies to both, only the CLI name (bao) and the environment variables (BAO_ADDR) differ. The container image softens the transition further: inside it, `/usr/bin/vault` is a symlink to `/usr/bin/bao`, so `vault status` in the pod shell does the same as `bao status` – and `vault version` answers `OpenBao v2.6.2`. Old scripts and habits keep working; this guide nevertheless writes `bao` throughout.

If you want to understand the concepts – seal/unseal, auth methods, policies, secrets engines, leases – from the ground up, with labs that run on a laptop, that is in my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice). This guide assumes those concepts and shows how they come together in a cluster.

The building blocks

Component	Version	Role
Ubuntu Server	24.04 LTS	the operating system of the six nodes
RKE2 Kubernetes	v1.36.4+rke2r1	3 control-plane nodes, 3 workers, Cilium as CNI, Traefik as ingress, Longhorn as storage
OpenBao Helm chart openbao/openbao	chart 0.29.4, app 2.6.2	The deployment
Helm	3.16	Installs the chart
bao CLI	2.6.2	Included in the pod; local install optional

The repository

The configuration files for the cluster (`rke2/`) and the values file that Part I explains piece by piece (`k8s/values.yaml`) are not printed in full in the text; they live in the repository for this series. The reason is the PDF: shell commands can be copied out of it, YAML and policies cannot – leading spaces are not characters in a PDF, only position, and they are gone when you copy. Everything that depends on indentation therefore comes from the repository. Every relative path in this guide means the directory of this note. So clone first and change into it:

```
git clone https://github.com/thomaszachmann/field-notes.git
cd field-notes/openbao/01-openbao-on-kubernetes
ls rke2/ k8s/ openbao/ commands.sh
```

All `scp`, `helm` and `kubectl` calls in the following parts run from here. And if you would rather not copy the commands from the PDF either (some viewers break words with underscores apart when copying): `commands.sh` is Appendix A as a file.

The cluster

This guide assumes a running Kubernetes cluster. Here it is RKE2, Rancher's Kubernetes distribution: six nodes running Ubuntu Server 24.04 LTS on one physical host, three of them control plane (RKE2 calls them *servers*), three workers (*agents*). If you already have a cluster with an ingress class and a StorageClass, skip this section and adjust `ingressClassName` and `storageClass` in the values later.

The setup in short, so that it is clear where the cluster comes from – the full guide is at docs.rke2.io/install/ha. All commands run as root on the machine in question.

1. The first server. The configuration lives in `/etc/rancher/rke2/config.yaml`; the installer reads it on start. The file is in the repository as `rke2/config-server-1.yaml`:

```
token: <cluster-token>
cni: cilium
node-taint:
  - "CriticalAddonsOnly=true:NoExecute"
```

Copy it from the workstation to the server, fill in the token there, install:

```
ssh root@<first-server> mkdir -p /etc/rancher/rke2
scp rke2/config-server-1.yaml root@<first-server>:/etc/rancher/rke2/config.yaml
ssh root@<first-server>
# on the server: replace the placeholder with the token
vi /etc/rancher/rke2/config.yaml
curl -sL https://get.rke2.io | sh -
systemctl enable --now rke2-server.service
```

token is the secret every further node joins with – chosen freely, for example generated with `openssl rand -hex 32`, and into the password manager like the unseal keys later. Leave it out and RKE2 generates one itself and stores it under `/var/lib/rancher/rke2/server/node-token`. A fixed name for the API server (`tls-san`) is only needed once a VIP or a DNS name is to sit in front of the three servers; in a lab the first server's IP, which is in the certificate anyway, is enough. The `node-taint` keeps workloads off the control-plane nodes – and is the reason Traefik only runs on the workers in Part IV.

`cni: cilium` picks the network plugin. Without the line RKE2 takes Canal; both enforce NetworkPolicy (N° 7 relies on it), Cilium does it with eBPF and can add Hubble and the kube-proxy replacement later (docs.rke2.io/networking/basic_network_options). The choice is made before the first start: changing the CNI on a running cluster is not an upgrade, it is a rebuild.

The first start takes a few minutes while RKE2 pulls its images. `journalctl -u rke2-server -f` shows the progress; it is done when `/etc/rancher/rke2/rke2.yaml` exists. If the unit fails immediately, the reason is in the same output – `yaml: line N: could not find expected ':'` means the configuration file is broken, usually by lost indentation.

2. The two other servers. The same configuration plus the address of the first server (`rke2/config-server-n.yaml`), then the same installer and the same systemd unit:

```
server: https://<first-server-ip>:9345
token: <cluster-token>
cni: cilium
node-taint:
  - "CriticalAddonsOnly=true:NoExecute"
```

Three servers form an etcd quorum: one may fail.

3. The workers. Only server and token (rke2/config-agent.yaml) – CNI and taints come from the servers – and the installer as agent:

```
server: https://<first-server-ip>:9345
token: <cluster-token>
```

```
curl -sL https://get.rke2.io | INSTALL_RKE2_TYPE="agent" sh -
systemctl enable --now rke2-agent.service
```

4. kubectl and helm on the workstation. The server itself has a kubectl under /var/lib/rancher/rke2/bin/, but the work is done from your own machine. kubectl may differ from the cluster by at most one minor version (so 1.35 to 1.37 here), helm is 3.16 or newer:

```
# macOS
brew install kubectl helm

# Linux (amd64)
KUBECTL_VERSION=$(curl -Ls https://dl.k8s.io/release/stable.txt)
curl -LO "https://dl.k8s.io/release/${KUBECTL_VERSION}/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 \
  | bash

kubectl version --client
helm version
```

The official guides, also for Windows and other architectures: kubernetes.io/docs/tasks/tools and helm.sh/docs/intro/install.

5. The kubeconfig. The first server writes it to /etc/rancher/rke2/rke2.yaml, with 127.0.0.1 as the address. Copy it to your own machine and replace the address with the first server's IP:

```
mkdir -p ~/.kube
scp root@<first-server>:/etc/rancher/rke2/rke2.yaml ~/.kube/config
chmod 600 ~/.kube/config
kubectl config set-cluster default --server=https://<first-server-ip>:6443
kubectl get nodes
```

All six nodes should be Ready, the three servers with the roles control-plane,etcd,master. And Cilium runs as a DaemonSet on every node:

```
kubectl get pods -n kube-system -l k8s-app=cilium
```


6. Ingress and storage. Since RKE2 v1.36, Traefik is the default ingress controller; it arrives as a DaemonSet on the workers with nothing further to do. Longhorn is added via Helm – once, for the whole cluster. It runs as a DaemonSet on the nodes that allow it: because of the taint from step 1, only on the three workers, and that is where the replicas live too (default: three, one per worker). First, open-iscsi on those three nodes, because Longhorn attaches volumes to the node over iSCSI. On Ubuntu 24.04:

```
# on every worker, as root
apt-get update && apt-get install -y open-iscsi
systemctl enable --now iscsid
```

Then from the workstation:

```
helm repo add longhorn https://charts.longhorn.io
helm repo update
helm upgrade --install longhorn longhorn/longhorn \
  --namespace longhorn-system --create-namespace
kubectl get storageclass
```

After that the StorageClass longhorn exists, which the values in Part I require. And Longhorn has landed where it belongs:

```
# expected: three pods, one per worker
kubectl get pods -n longhorn-system -l app=longhorn-manager -o wide
# expected: the three workers, Schedulable true
kubectl get nodes.longhorn.io -n longhorn-system
```

7. The way in from outside. Everything so far lives inside the cluster. A browser or the bao CLI on the workstation, however, must reach OpenBao from outside, and for that three things are missing: a **name** under which OpenBao is reachable, a **certificate** for that name, and a **place** that forwards requests for that name into the cluster.

That place is a reverse proxy on its own small VM in front of the cluster. It holds the certificate and terminates TLS; everything behind it speaks plain HTTP. That is the reason for `tlsDisable: true` in the values of Part I: OpenBao itself never sees a certificate. The path of a request:

```
Browser —https→ reverse proxy —http→ worker:80 (Traefik) → Ingress → svc/openbao:8200
```

Why port 80 of a **worker**: Traefik runs as a DaemonSet with hostPort 80/443, but because of the taint from step 1 only on the workers. So the proxy points at the IP of one worker; if that one goes down, the UI is gone although OpenBao keeps running. What that means, and why it is accepted in a lab, is in Part IV.

Traefik picks the Ingress by the Host header. The name the proxy forwards therefore has to be the same as in `server.ingress.hosts` of the values – in this guide the placeholder `bao.example.internal` stands for it. How name, certificate and proxy came about here – Nginx Proxy Manager, DuckDNS, Let's Encrypt – is in Appendix C, step by step. If you already have a proxy, you only enter the name and the worker IP there.

Whether the path up to Traefik works can be checked right now, before OpenBao is installed:

```
curl -s -o /dev/null -w '%{http_code}\n' \
-H 'Host: bao.example.internal' http://<worker-ip>/
```

404 is correct here: Traefik answers but does not know the host yet. Once the Ingress from Part I exists, this becomes OpenBao's answer.

On kind instead of RKE2

If you do not have six machines but a laptop with Docker: Parts I to VII also run on [kind](#) – Kubernetes inside a Docker container. What is missing is exactly what was set up above: Longhorn, Traefik, the reverse proxy – and instead of Cilium, kind brings its own CNI (kindnet, which since 0.24 enforces NetworkPolicy as well). The replacement, carried out with kind v0.32.0:

A cluster with one node whose port 80 is mapped to the laptop, and ingress-nginx on top – that is the route the kind documentation prescribes for ingress:

```
cat > kind-config.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
EOF
kind create cluster --name bao --config kind-config.yaml
kubectl apply -f https://kind.sigs.k8s.io/examples/ingress/deploy-ingress-nginx.yaml
```

```
$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
bao-control-plane   Ready    control-plane   2m13s   v1.36.1

$ kubectl get storageclass
NAME                PROVISIONER          RECLAIMPOLICY   VOLUMEBINDINGMODE
standard (default)  rancher.io/local-path Delete           WaitForFirstConsumer

$ kubectl get ingressclass
NAME    CONTROLLER          PARAMETERS
nginx   k8s.io/ingress-nginx <none>
```

No Longhorn: kind ships standard (local-path) as the default StorageClass. No Traefik, no proxy: ingress-nginx listens on port 80 of the node container, and that sits on localhost. As hostname, bao.127.0.0.1.nip.io is enough – nip.io resolves any name with an IP in it to exactly that IP, with no DNS record and no Appendix C.

That changes three values in k8s/values.yaml. Instead of touching the file, they go onto the helm call in Part I as overrides:

```
helm upgrade --install openbao openbao/openbao \
  --version 0.29.4 \
  --namespace openbao --create-namespace \
  --values k8s/values.yaml \
  --set server.dataStorage.storageClass=standard \
  --set server.ingress.ingressClassName=nginx \
  --set 'server.ingress.hosts[0].host=bao.127.0.0.1.nip.io'
```

After that it looks like Part I, only with different names in the CLASS and STORAGECLASS columns:

```
$ kubectl get sts,ingress,pvc -n openbao
NAME                                READY   AGE
statefulset.apps/openbao           0/1     73s
```

NAME	CLASS	HOSTS	ADDRESS	PORTS
ingress.networking.k8s.io/openbao	nginx	bao.127.0.0.1.nip.io	localhost	80

NAME	STATUS	CAPACITY	STORAGECLASS
persistentvolumeclaim/data-openbao-0	Bound	10Gi	standard

And from outside, without TLS – the listener speaks HTTP, and this time no proxy in front changes that:

```
$ export BAO_ADDR=http://bao.127.0.0.1.nip.io
$ bao status
Key                Value
---                -
Seal Type          shamir
Initialized        false
Sealed             true
Version            2.6.2
Storage Type       raft
HA Enabled         true
```

From here on Parts II to VII apply as printed: `init` and `unseal` in the pod shell, Kubernetes auth via `TokenReview` (the chart renders the `ClusterRoleBinding` to `system:auth-delegator` here too), snapshots via `kubectl cp`. Three things read differently: `https://bao.example.internal` is `http://bao.127.0.0.1.nip.io` throughout; “worker” in Part IV means the one node on kind; and `storageClass: longhorn` on the audit volume in Part VI becomes `standard` as well. And: `kind delete cluster --name bao` takes the PVC with it. Snapshots are no less of a duty on a laptop, only more quickly forgotten.

Part I – Helm

The chart

OpenBao maintains its own chart, derived from the Vault chart. It knows three modes: dev (in-memory, unsealed – for playing only), standalone (one pod, persistent volume) and ha (several pods with Raft quorum). This guide takes standalone.

```
helm repo add openbao https://openbao.github.io/openbao-helm
helm repo update
helm search repo openbao/openbao --versions | head -3
```

Why one pod and not three

Three workers, Longhorn, a chart that can do ha – the question is obvious. What three pods give you: if the active pod or its worker dies, a standby takes over within seconds, and upgrades proceed pod by pod without downtime. For everything that depends on OpenBao continuously – the External Secrets Operator from N° 2, cert-manager from N° 5 – that would be the real gain. What they do not give you: protection against the failure of the one physical host. Six VMs on one machine are six pods on one machine.

And what they cost as long as the unseal keys are typed by hand (Part II): every pod has its own seal. After a power cut that is nine key entries instead of three – and until all three are unsealed there is no quorum and therefore no OpenBao at all. A standby that sits sealed after a restart takes over nothing. With Shamir, ha is therefore not more robust than standalone but more fragile. On top of that: three PVCs, each replicated three times by Longhorn, so nine copies of the same data; and the `service_registration` block this guide drops below because of its 403 warnings becomes necessary so that the Service finds the *active* pod.

ha pays off once two things are in place: **auto-unseal**, so that all three pods unseal themselves (N° 6: transit seal against the VM OpenBao), and **something that notices the outage**. Both come later in the series. Until then one pod is the more honest choice – it is sealed just as often, but only once.

The values

The full file (`k8s/values.yaml`) is 70 lines; here are the parts with their reasoning.

```
global:
  tlsDisable: true
```

TLS is terminated at the reverse proxy. The chart uses this switch in several places: it sets `BAO_ADDR` inside the pod to `http://`, configures the probes without TLS and omits the `tls` block from the ingress. Setting only the listener to `tls_disable = 1` but not `global.tlsDisable` leaves the readiness probes pointing nowhere.

```
injector:
  enabled: false
```

In the end, an application in a pod is supposed to use a secret from OpenBao. The application itself knows nothing about OpenBao – so something has to fetch the secret and put it within reach. There are two common ways to do that, and this switch decides about the first one.

The **injector** is a helper program the chart can install alongside (it comes from the Vault ecosystem, the `vault-k8s` project). It steps in at the moment Kubernetes creates a new pod: if the pod carries a certain marker (an annotation such as `vault.hashicorp.com/agent-inject: "true"`), the injector quietly rebuilds the pod and places a second, small container next to it – a *sidecar*. That sidecar logs in to OpenBao, fetches the secret and writes it as a file into a directory both containers can see. The application then simply reads a file.

The **External Secrets Operator** (ESO) takes a different route: it runs once in the cluster, fetches secrets from OpenBao and stores them as perfectly ordinary Kubernetes Secrets. The application gets them like any other Secret – as an environment variable or as a file – and the pod stays untouched.

This guide and the ones that follow (Nº 2, Nº 4) take the second route. That is why the injector stays off: it would only run without being used – and a program that is allowed to rebuild every new pod is one you do not want in the cluster without a reason.

```
server:
  standalone:
    enabled: true
    config: |
      ui = true

      listener "tcp" {
        address      = "[::]:8200"
        cluster_address = "[::]:8201"
        tls_disable   = 1
      }

      storage "raft" {
        path = "/openbao/data"
      }
```

The most important intervention is storage `"raft"`. The chart default for standalone is storage `"file"` – a backend that **cannot take snapshots**. `bao operator raft snapshot save` only exists with Raft. Whoever keeps the default has no way to take a consistent backup other than copying the PVC – and that is not consistent while the process is running.

`api_addr` and `cluster_addr` are deliberately absent: the chart sets `BAO_API_ADDR` and `BAO_CLUSTER_ADDR` as environment variables on the StatefulSet, also in standalone mode, and OpenBao reads them at startup.

`service_registration "kubernetes"` was in this block in the original setup – and is deliberately **no longer** part of `k8s/values.yaml`. The registration is meant to write the state (`active`, `sealed`, ...) as labels onto the pod so that an HA service selects only the active node. In standalone mode, however,

the chart renders neither the Role nor the RoleBinding for that, and OpenBao then logs every five seconds:

```
[WARN] service_registration.kubernetes: unable to set initial state due; will retry:
err="GET https://10.43.0.1:443/api/v1/namespaces/openbao/pods/openbao-0 ..."
resp statusCode: 403"
```

For 22 hours before anyone noticed. In standalone mode the services select on component: server with publishNotReadyAddresses: true anyway – the labels would change nothing. So leave it out, or add a Role with get/patch on pods if you want the labels for monitoring.

```
dataStorage:
  enabled: true
  size: 10Gi
  storageClass: longhorn

persistentVolumeClaimRetentionPolicy:
  whenDeleted: Retain
  whenScaled: Retain
```

Both explicit, although they would be the defaults. The reason is the same as for ingressClassName: a changed cluster default must not silently move the Raft data elsewhere or take it along on helm uninstall.

```
ingress:
  enabled: true
  ingressClassName: traefik
  hosts:
    - host: bao.example.internal
      paths: []

ui:
  enabled: true
  serviceType: ClusterIP
```

No tls block, no cert-manager. The reverse proxy in front of the cluster holds the certificate and forwards to Traefik over HTTP.

Installing

```
helm upgrade --install openbao openbao/openbao \
  --version 0.29.4 \
  --namespace openbao --create-namespace \
  --values k8s/values.yaml
```

Without --wait. The chart's readiness probe is bao status, which fails as long as OpenBao is sealed. On a fresh installation the pod is only *Ready* after init + unseal – --wait would just run into its timeout.

Check beforehand without changing anything:

```
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
--namespace openbao --values k8s/values.yaml --dry-run=server
```

Afterwards:

```
$ kubectl get sts,svc,ingress,pvc -n openbao
NAME                                READY   AGE
statefulset.apps/openbao            0/1     1m

NAME                                TYPE             CLUSTER-IP      PORT(S)
service/openbao                     ClusterIP         10.43.179.240    8200/TCP,8201/TCP
service/openbao-internal             ClusterIP         None             8200/TCP,8201/TCP
service/openbao-ui                   ClusterIP         10.43.158.186    8200/TCP

NAME                                CLASS    HOSTS                PORTS
ingress.networking.k8s.io/openbao   traefik  bao.example.internal  80

NAME                                STATUS    CAPACITY   STORAGECLASS
persistentvolumeclaim/data-openbao-0 Bound     10Gi       longhorn
```

READY 0/1 is correct here. The pod runs, OpenBao answers, but it is not initialised – and the chart rightly calls that “not ready”.

What `helm upgrade` does *not* do later

Normally in Kubernetes, when a pod’s template changes – a different image, a different environment variable, a new configuration – Kubernetes replaces the running pod with a new one by itself. For a StatefulSet that is called a *rolling update*, and it happens right after `helm upgrade`, without asking.

For OpenBao that would be a problem. Every new pod starts **sealed** (Part II): it runs, but answers nothing until someone enters three unseal keys. A rolling update at three in the morning would be an outage until morning. That is why the chart sets the StatefulSet to `updateStrategyType: OnDelete`. It means: Kubernetes records the new template but leaves the running pod alone. Only when you delete the pod yourself does the new one appear – with the new configuration, and sealed.

This is what it looks like when you check after a `helm upgrade` with changed values:

```
$ kubectl get sts -n openbao openbao \
-o jsonpath='{.spec.template.spec.containers[0].resources.requests.cpu}'
150m
$ kubectl get pod -n openbao openbao-0 \
-o jsonpath='{.spec.containers[0].resources.requests.cpu}'
100m
```

The StatefulSet has the new value, the pod still the old one. The change takes effect when you want it to – and only then:

```
kubectl -n openbao delete pod openbao-0      # pod comes back, sealed
kubectl exec -it -n openbao openbao-0 -- bao operator unseal # x3
```

It is the same trade-off as a configuration restart on a VM: a restart costs an unseal, so it only happens on purpose. Whoever forgets to delete the pod after a helm upgrade keeps running with the old configuration – and wonders why the change does nothing (Part VII).

Part II – init and unseal

Why this is not automated

`bao operator init` generates the unseal keys and the root token. That output appears **exactly once**. If `init` runs in a pipeline, in Ansible or in an AI chat, the key material ends up in logs, in task output or in a context window. Hence: a normal terminal, a shell inside the pod, output straight into the password manager, then close the terminal.

init

```
kubectl exec -it -n openbao openbao-0 -- sh
/ $ bao operator init -key-shares=5 -key-threshold=3
```

Shamir 5 of 3: five keys, three suffice to unseal. In a lab where one person holds all five, that is no security gain over 1/1 – but it is the format a later move to several custodians expects, and it costs nothing.

The output contains `Unseal Key 1` to `5` and `Initial Root Token`. All of it into the password manager. **Not** into this repo, **not** onto the NAS where the snapshots will later live – the snapshots are encrypted with exactly these keys, and whoever keeps both in one place has lost both when that place is gone.

unseal

```
/ $ bao operator unseal      # enter key, no echo
/ $ bao operator unseal
/ $ bao operator unseal
/ $ bao status
```

Key	Value
---	-----
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	5
Threshold	3
Version	2.6.2
Storage Type	raft
HA Enabled	true
HA Cluster	https://openbao-0.openbao-internal:8201
HA Mode	active

`HA Enabled true` on a single pod is surprising – it is a property of Raft, not of the operating mode. The pod is the active (and only) node.

Without an argument, `bao operator unseal` prompts for the key interactively and reads it without echo. A key passed as an argument would sit in the shell history and be visible to other processes via `/proc`.

After the unseal the pod switches to READY 1/1.

What happens after every restart

The pod dies, gets rescheduled, the node is patched: OpenBao starts **sealed**. That is the deliberately accepted price of the Shamir decision. As long as nothing depends on it, it is merely inconvenient. Once the External Secrets Operator hangs off it (N° 2), every restart is an outage for everything that has to renew secrets. At that point auto-unseal (transit engine on the VM OpenBao, or a cloud KMS) belongs on the list.

The first login

```
/ $ bao login  
Token (will be hidden):
```

The root token. From here the shell is root in OpenBao – and `bao login` writes the token to `~/.vault-token` inside the container. More on that in Part V; for now, this is the state in which the configuration is done.

Part III – Kubernetes auth

How it works

Every pod in Kubernetes carries an ID card: the token of its ServiceAccount. Kubernetes places it into the pod as a file on start (`/var/run/secrets/kubernetes.io/serviceaccount/token`), renews it regularly, and it states which ServiceAccount in which namespace the pod is. The pod has it without anyone having to hand it over.

The login uses exactly this ID card. Four steps:

1. The pod sends its token to OpenBao (`auth/kubernetes/login`) and names a role, for example `demo`.
2. OpenBao cannot verify the token itself – it did not issue it. So it asks the one who can: the Kubernetes API server. There is a dedicated API for that, `TokenReview`: “Is this token genuine, and whose is it?”
3. The API server answers: genuine, ServiceAccount `demo` in namespace `demo`.
4. OpenBao compares that with the role. If ServiceAccount and namespace match what the role says, it issues an OpenBao token – with the policies the role names. If they do not match, the answer is 403.

The point: nowhere did a password or a key have to be handed out. The pod already had its ID card, and for the question to the API server OpenBao needs only what it has as a pod itself – its own ServiceAccount token and the cluster’s CA certificate.

Enable and configure

```
/ $ bao auth enable kubernetes
/ $ bao write auth/kubernetes/config \
    kubernetes_host="https://kubernetes.default.svc:443"
```

Nothing more. Because OpenBao itself runs in the cluster, it uses its own ServiceAccount token and the CA from `/var/run/secrets/kubernetes.io/serviceaccount/` for the `TokenReview`. The chart provides the permission for that: it renders a `ClusterRoleBinding` of the ServiceAccount `openbao` to the built-in `ClusterRole` `system:auth-delegator` – exactly the right to submit `TokenReviews`, and nothing else. If OpenBao runs **outside** the cluster, `kubernetes_ca_cert` and `token_reviewer_jwt` must be set explicitly – and the reviewer token needs that same `ClusterRole`.

A role for one consumer

A role binds a ServiceAccount to policies:

```
/ $ bao write auth/kubernetes/role/demo \  
    bound_service_account_names=demo \  
    bound_service_account_namespaces=demo \  
    token_policies=default \  
    token_ttl=1h
```

```
/ $ bao read auth/kubernetes/role/demo  
Key                               Value  
---                               -  
alias_name_source                 serviceaccount_uid  
bound_service_account_names      [demo]  
bound_service_account_namespaces [demo]  
token_policies                   [default]  
token_ttl                        1h
```

alias_name_source: serviceaccount_uid means: the identity in OpenBao hangs on the UID of the ServiceAccount. If the ServiceAccount is deleted and recreated, it is a new identity from OpenBao's point of view – the old alias remains as a corpse in the identity store until someone removes it.

default is the policy every token gets anyway – it allows little more than looking at one's own token. For this guide that is enough: the role says *who* may log in and *which* policy they get. What a real consumer may read is decided when there is one.

Checking without ESO

The quickest test is a pod with the ServiceAccount that performs the login by hand:

```
kubectl create namespace demo  
kubectl create serviceaccount demo -n demo  
kubectl run -n demo --rm -it --restart=Never \  
    --overrides='{ "spec": { "serviceAccountName": "demo" } }' \  
    --image=curlimages/curl:8.11.1 probe -- sh
```

The pod pretends to be the application that comes later. The part that matters is `--overrides`: `kubectl run` no longer has a flag for the ServiceAccount, so the field is laid over the generated manifest as a JSON fragment. Without that line the pod would run as default – and the role `demo` would reject it no matter how correct everything else is. `--restart=Never` makes sure a single pod is created rather than a Deployment; `--rm` removes it after exit; `-- sh` replaces the image's entrypoint with a shell. The `curl` image is pinned on purpose so the test still does the same thing a year from now.

Inside the pod:

```
JWT=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)  
curl -s http://openbao.openbao.svc:8200/v1/auth/kubernetes/login \  
-d '{"role":"demo","jwt":"'$JWT'" }' | head -c 400
```

This is the login ESO will perform automatically later – done by hand here. The first line reads the token Kubernetes mounts into every pod for its ServiceAccount: a signed JWT carrying the ServiceAccount's namespace and name. The second sends it to the auth method – without a token of its own, because

the login path is unauthenticated; the response *is* the token. `openbao.openbao.svc` is the cluster DNS name of the Service (name.namespace.svc), port 8200 the listener; pods have no need for the Ingress. The escapes in the `-d` body are there because only double quotes expand `$JWT`. `head -c 400` truncates the response so the whole token and its accessor do not end up in the scrollbar.

A `client_token` in the response means: auth method, TokenReview and role work. A 403 permission denied almost always means: namespace or name of the ServiceAccount do not match the role – not that the policy forbids something, because the policy only comes into play after the login.

Part IV – Access from outside

The path

```
Browser —https→ reverse proxy (certificate) —http→ worker node:80 (Traefik hostPort)
                                     |
                                     ▼
                               Ingress openbao → svc/openbao:8200
```

In RKE2, Traefik runs as a DaemonSet with hostPort 80/443 – but only on the workers, because the control-plane nodes are tainted and the DaemonSet does not tolerate the taints. The reverse proxy's target is therefore the IP of one worker. If that worker goes down, the UI is unreachable although OpenBao keeps running. The proxy itself – Nginx Proxy Manager, DuckDNS, Let's Encrypt – is described in Appendix C.

A floating VIP (MetalLB) would solve that. It was deliberately not installed: the availability of the *UI* is not worth another controller in a lab – the workloads in the cluster reach OpenBao via `svc/openbao`, and that does not depend on any particular worker.

The bao CLI from outside

```
export BAO_ADDR=https://bao.example.internal
bao status
bao login    # token
```

Two traps that cost time while debugging:

- A terminal with `BAO_ADDR` pointing at the ingress but no token: **every** call answers 403 permission denied – even `bao token capabilities`. It looks like a policy problem and is a missing `login.bao token lookup` first.
- `bao login` writes to `~/.vault-token` – the same path the Vault CLI uses. Whoever runs a Vault or a second OpenBao in parallel overwrites one system's token when logging in to the other. A puzzling 403 is then usually a token from the other system.

The UI

`https://bao.example.internal/ui/` – login with the root token or, after hardening, by username. The UI is handy for an overview of mounts, policies and leases; for anything reproducible the CLI is better, because its calls can move into a script or into OpenTofu.

Part V – The current state, honestly

This is where this guide falls behind the VM setup (N° 3). On the VM the bootstrap sequence has been carried out in full and verified. In the cluster it has **not**. This is what it looks like:

```
/ $ bao audit list
No audit devices are enabled.

/ $ bao auth list
Path          Type          Description
----          -
kubernetes/   kubernetes    n/a
token/        token         token based credentials

/ $ bao policy list
default
root

/ $ bao token lookup | grep -E 'display_name|policies'
display_name    root
policies        [root]

/ $ ls -la ~/.vault-token
-rw----- 1 openbao openbao /home/openbao/.vault-token
```

Four findings:

- 1. No audit device.** Nobody can answer who read which path when. For a secrets store, that is the main reason to run one at all.
- 2. The root token is the only administrative access.** There is no userpass, no admin policy. The root token never expires and cannot be scoped.
- 3. The root token sits as a file in the pod.** `bao login` wrote it to `/home/openbao/.vault-token`. Anyone with `kubectl exec` on the pod – that is, anyone with sufficient RBAC in the namespace `openbao` – is root in OpenBao. The file lives in the container filesystem and disappears on restart; until then it is there.
- 4. No snapshots.** The PVC is the only copy of the Raft data. Longhorn does replicate the volume across nodes, but that protects against a disk failure, not against an accidental `helm uninstall` with a deleted PVC, a broken upgrade or a slip with `bao delete`.

Acceptable for a demo, for nothing else. Part VI describes what to do – the commands come from the VM runbook (N° 3) and were run through for this note on a kind cluster (see “The cluster”). Two of them no longer work on OpenBao 2.6 the way they do on the VM; Part VI shows which and why.

Part VI – Hardening: what comes next

The order is not arbitrary. Audit first, so that everything after is logged. Root token last, and only once the replacement access has been **proven**.

1. Remove the token file from the pod

Immediately, independent of the rest:

```
/ $ rm ~/.vault-token
```

For further work, hold the token only in the environment variable of the running shell:

```
/ $ read -rs BAO_TOKEN; export BAO_TOKEN
```

`read -rs` reads without echo and without a history entry. When the shell is closed, the token is gone.

2. Audit device

The obvious command – still the right one on the VM in N° 3 – no longer works on 2.6.2:

```
$ bao audit enable file file_path=/openbao/audit/audit.log
Error enabling audit device: Error making API request.
Code: 400. Errors:

* cannot enable audit device via API; use declarative, config-based audit device management instead
```

Since OpenBao 2.3.2, creating audit devices via the API is off by default (`unsafe_allow_api_audit_creation = false`). The reason is a good one: a file device writes to an arbitrary path, a socket device to an arbitrary socket – whoever captured an admin token could use the server as a writing tool. Since OpenBao 2.4.0, audit devices therefore belong in the server configuration as an audit block – the same HCL file that already holds listener and storage. On the VM that would be `/etc/openbao/openbao.hcl`; in the cluster the Helm chart writes this file from the value `server.standalone.config` into a ConfigMap and mounts it at `/openbao/config`. Both version numbers are OpenBao versions, not those of the chart (0.29.4) or of RKE2. Two additions to the values – the block, and the volume it writes to (in the repository as `k8s/values-hardened.yaml`, passed in addition to `k8s/values.yaml`):


```

server:
  standalone:
    config: |
      # ... ui, listener and storage as in Part I ...

      audit "file" "audit" {
        options = {
          file_path = "/openbao/audit/audit.log"
        }
      }

  auditStorage:
    enabled: true
    size: 2Gi
    storageClass: longhorn

```

/openbao/audit is the path the chart provides as a separate PVC with auditStorage – without that volume the log sits in the container and is gone after a restart. A device from the configuration can neither be changed nor deleted via the API; if the block disappears, the device disappears on the next restart.

The second hurdle: `helm upgrade` with these values **fails**.

```

Error: UPGRADE FAILED: StatefulSet.apps "openbao" is invalid: spec: Forbidden:
updates to statefulset spec for fields other than 'replicas', 'ordinals',
'template', 'updateStrategy', 'revisionHistoryLimit',
'persistentVolumeClaimRetentionPolicy' and 'minReadySeconds' are forbidden

```

A second volume is a second volumeClaimTemplate, and that is immutable on an existing StatefulSet. The way forward is a new StatefulSet – and that is exactly what whenDeleted: Retain in Part I is for:

```

kubectl delete sts -n openbao openbao      # PVC data-openbao-0 stays (Retain)
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  --namespace openbao --values k8s/values.yaml --values k8s/values-hardened.yaml
kubectl exec -it -n openbao openbao-0 -- bao operator unseal    # x3

```

```

$ kubectl get pvc -n openbao
NAME                STATUS    CAPACITY
audit-openbao-0     Bound    2Gi
data-openbao-0      Bound    10Gi

$ bao audit list
Path      Type      Description
----      -
audit/    file      n/a

```

The chart creates the StatefulSet afresh, the pod finds its data PVC again, and OpenBao enables the device on start – the log says `core: enabled audit backend: path=audit/ type=file`. If you are setting up from scratch, pass both values files from the start and skip the delete; `k8s/values.yaml` on its own deliberately shows the state *before* hardening.

Two things to know: if OpenBao cannot write to the audit log, it **refuses requests**. A full volume takes the secrets store off the network. That is a security feature and, without rotation, a time bomb – on the VM logrotate handles it; in the cluster it takes a sidecar or a second audit device (syslog, socket to

a log collector) that becomes the primary sink. And: from now on all entries show Traefik's IP instead of the real client, as long as `x_forwarded_for_authorized_addrs` is not set on the listener.

3. Admin policy and userpass

Step 5 revokes the root token. Before that there has to be another way in – one that expires, is tied to a person and shows up in the audit log by name. That takes two things: a **policy** admin that says what this access may do, and a **userpass login** admin that receives a token with exactly this policy on login (the policy is also in the repository as `openbao/admin.hcl`):

```
/ $ bao policy write admin - <<'EOF'
path "sys/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "auth/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "identity/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}
EOF

/ $ bao auth enable userpass
/ $ bao write auth/userpass/users/admin \
  password="$(read -rs -p 'Password: ' p; echo "$p")" \
  token_policies=admin token_ttl=1h token_max_ttl=8h
```

Three lines, and they are deliberately broad: `sys/*` is administration – mounts, auth methods, policies, audit, snapshots and the way to a new root token (`sys/generate-root-token/*`, step 4). `auth/*` and `identity/*` are users, roles and identities. What is missing is missing on purpose: data paths. The admin manages OpenBao, it does not read secrets. A long list of individual `sys/` paths would gain nothing – whoever may write policies can grant themselves the rest. The gain over the root token is not less power but **expiry, accountability and a password that can be rotated**.

The password belongs in the password manager **before** step 5. Under OpenBao it is recovery material, not a convenience: since 2.5.3 the unauthenticated `sys/generate-root/*` endpoints are off by default, and since 2.6.0 `bao operator generate-root` uses the authenticated `sys/generate-root-token` endpoints (an older CLI still talks to the old ones and gets a 405 – see Part VII). Three unseal keys alone are therefore **no way back** – it takes an additional login whose policy includes `sys/*`.

4. Prove the replacement access – do not skip

```
# -token-only: otherwise 'bao login' overwrites the running root token in
# ~/.vault-token BEFORE it is proven that the new access works.
/ $ ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"

/ $ BAO_TOKEN="$ADMIN_TOKEN" bao token lookup | grep policies
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao policy list

# The break-glass path itself: -init starts a root generation, -cancel
# aborts it. That proves the capability without producing a root token
# you would then have to handle.
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel

/ $ unset ADMIN_TOKEN
```

Only once admin appears in the policies, `policy list` works **and** the `generate-root -init/-cancel` pair goes through is the replacement access proven. If any of it fails: do **not** run step 5.

5. Revoke the root token

```
/ $ bao token lookup | grep policies      # [root]
/ $ bao token revoke -self
/ $ bao token lookup                      # must fail now
```

From here `userpass` is the way in. Break glass if that access is lost:

```
bao login -method=userpass username=admin
bao operator generate-root -init
bao operator generate-root          # 3x with unseal keys
```

6. A first snapshot

Finding 4 from Part V: the PVC is the only copy of the Raft data. Before the root token is gone, a snapshot belongs on a machine outside the cluster – two commands:

```
kubectl exec -n openbao openbao-0 -- \
  sh -c 'BAO_TOKEN=... bao operator raft snapshot save /tmp/bao.snap' && \
kubectl cp openbao/openbao-0:/tmp/bao.snap ./openbao-$(date +%Y%m%dT%H%M).snap
```

And the check OpenBao itself does not offer (`raft snapshot inspect` only exists in Vault): the archive is a gzipped tar with four entries –

```
tar -tzf openbao-*.snap
tar -xz0f openbao-*.snap SHA256SUMS
tar -xz0f openbao-*.snap state.bin | sha256sum      # must match the line above
```

```
meta.json
state.bin
SHA256SUMS
SHA256SUMS.sealed
30415d1f6fc46454441004a9c4e01eaf764c53a0051cac3fcc93eae288223249 meta.json
ffcf434b4ad464420a558a883706f929306f7df966b948bc41dd2268a87d3047 state.bin
ffcf434b4ad464420a558a883706f929306f7df966b948bc41dd2268a87d3047 -
```

This is a snapshot, not a backup: it sits on your machine, nobody takes it regularly, and nobody has rehearsed the restore. The CronJob with its own identity, the object storage and the restore drill are N° 7.

This is where hardening by hand ends. Everything from Parts III and VI is state that is gone on a rebuild; capturing it as code – policies, auth methods, roles – is N° 9.

Part VII – What can go wrong

Pod stays 0/1 Running

Expected as long as OpenBao is sealed or not initialised. `kubectl exec ... bao status` shows which case it is. Only when `Sealed false` and the pod still does not become Ready is something broken – then look at the probe in the StatefulSet (`kubectl describe pod`).

Log full of `service_registration ... 403`

A warning every five seconds: `service_registration "kubernetes"` is configured, but the ServiceAccount may not patch its own pod. In standalone mode the chart does not render the required Role. Remove the block from the server config (see Part I) or create a Role with `get, update, patch` on pods and a RoleBinding to the ServiceAccount `openbao`.

`helm upgrade` changes nothing

`OnDelete`. Delete the pod, unseal. See Part I.

`403 permission denied on everything`

No token, expired token, or token from the wrong system. `bao token lookup` first. Only when that works and a specific path still returns 403 is it the policy – `bao token capabilities <path>` shows deny.

Kubernetes login fails

Message	Cause
<code>service account name not authorized</code>	<code>bound_service_account_names</code> does not match
<code>namespace not authorized</code>	<code>bound_service_account_namespaces</code> does not match
Post "https://kubernetes.default.svc:443/.../tokenreviews": ...	OpenBao cannot reach the API server, or its own token may not submit a TokenReview
<code>permission denied only when reading a path</code>	Login succeeded; the role's policy does not allow the path

OpenBao stops accepting requests, log mentions audit

The audit volume is full or not writable. OpenBao then refuses on purpose. Enlarge the volume or rotate the log – and afterwards set up a second audit device so that this does not happen again.

generate-root answers 405 unsupported operation

The CLI is older than 2.6.0 and still talks to `sys/generate-root/attempt`, which the server no longer serves since 2.5.3. The CLI in the pod always matches the server: `kubectl exec -it -n openbao openbao-0 -- sh`.

After node maintenance: sealed

Expected. `kubectl exec -it -n openbao openbao-0 -- bao operator unseal`, three times. If this happens too often: auto-unseal, see Part II.

Appendix A – All commands

The same list is in the repository as `commands.sh`.

```
# — Helm —————
helm repo add openbao https://openbao.github.io/openbao-helm
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  -n openbao --create-namespace -f k8s/values.yaml

# — init / unseal (shell in the pod, normal terminal) —————
kubectl exec -it -n openbao openbao-0 -- sh
bao operator init -key-shares=5 -key-threshold=3      # output -> password manager
bao operator unseal                                  # x3
bao status
read -rs BAO_TOKEN; export BAO_TOKEN                 # no 'bao login' in the pod

# — Kubernetes auth —————
bao auth enable kubernetes
bao write auth/kubernetes/config kubernetes_host="https://kubernetes.default.svc:443"
bao write auth/kubernetes/role/demo \
  bound_service_account_names=demo bound_service_account_namespaces=demo \
  token_policies=default token_ttl=1h

# — Hardening (keep the order) —————
# audit: k8s/values-hardened.yaml (audit block + auditStorage), then:
kubectl delete sts -n openbao openbao                # PVC stays (Retain)
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  -n openbao -f k8s/values.yaml -f k8s/values-hardened.yaml
bao operator unseal                                  # x3, then: bao audit list
bao policy write admin openbao/admin.hcl
bao auth enable userpass
bao write auth/userpass/users/admin password=... \
  token_policies=admin token_ttl=1h token_max_ttl=8h
ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel
bao token revoke -self                               # only after the test passed

# — Snapshot by hand —————
kubectl exec -n openbao openbao-0 -- \
  sh -c 'bao operator raft snapshot save /tmp/bao.snap'
kubectl cp openbao/openbao-0:/tmp/bao.snap ./openbao-$(date +%Y%m%dT%H%M).snap
tar -xz0f openbao-*.snap state.bin | sha256sum
```

Appendix B – Glossary

Seal / Unseal – OpenBao starts with encrypted storage and without the key to it. Unseal reassembles the master key from Shamir shares; until then OpenBao only answers status.

Shamir 5/3 – The master key is split into five shares, any three reconstruct it.

Raft – Integrated storage backend with a consensus protocol. Sensible even with one node, because only Raft can take snapshots.

Root token – The token from `init`. Valid indefinitely, cannot be scoped. Revoked after the bootstrap and regenerated via `generate-root` when needed.

Auth method – How a client proves its identity. `kubernetes` verifies ServiceAccount tokens via `TokenReview`; `userpass` verifies passwords.

Role (auth) – Binds identity attributes (ServiceAccount, namespace) to policies and token properties.

Policy – Allowed paths and capabilities. Default: everything denied.

Audit device – Logs every request and response (with hashed secrets). If OpenBao cannot write, it refuses requests. Defined in the server configuration since 2.4, not via the API.

TokenReview – The Kubernetes API by which a third party has a ServiceAccount token checked for validity and ownership.

Break glass – The route to a new root token when the old one is gone. Under OpenBao it needs unseal keys **and** a login with `sys/generate-root-token/*`.

OnDelete – Update strategy of the StatefulSet: new configuration only takes effect when the pod is deleted by hand.

Appendix C – The reverse proxy

Part IV assumes a reverse proxy in front of the cluster that holds the TLS certificate and forwards `bao.example.internal` to a worker. Here it is the **Nginx Proxy Manager** (NPM) in its own small VM, with a name from **DuckDNS** and a certificate from Let's Encrypt. If you already have a proxy – Caddy, Traefik on the host, a hand-written nginx – you do not need this appendix.

Why DuckDNS, and why DNS-01

The cluster is not reachable from outside, and that is how it should stay. A certificate from Let's Encrypt normally requires Let's Encrypt to reach the host on port 80 (HTTP-01) – that is out. The alternative is the DNS-01 challenge: Let's Encrypt checks a TXT record in the DNS zone, and for that the proxy must be allowed to write the zone. DuckDNS is a free dynamic DNS service whose API can do exactly that, and NPM ships it as a provider. The result: a genuine certificate for a name that points to a private IP, with no port open to the outside.

Two things worth knowing. First: the name is public. `<name>.duckdns.org` appears in the certificate transparency log as soon as the certificate is issued; the IP behind it is private, the name is not. Second: DuckDNS also resolves everything *below* your subdomain – `bao.<name>.duckdns.org` points to the same IP as `<name>.duckdns.org`. A wildcard certificate therefore covers every service that later ends up behind the proxy.

If you only want a quick test: `bao.10-0-0-20.sslip.io` resolves to `10.0.0.20` with no setup at all (`nip.io` likewise) – the IP in the name is that of the worker where Traefik holds port 80, i.e. the `<worker-ip>` from Part I; substitute your own address accordingly. DNS alone is not enough, though: Traefik routes by the Host header, and the Ingress only knows the name from `server.ingress.hosts`. So the `sslip` name has to go in there – the same `helm upgrade` as in Part I, just with this host; the pod does not need a restart for it. Without that, Traefik answers 404 page not found even though DNS and the connection are fine. There is no Let's Encrypt certificate for the name, because the IP is private and the zone is not yours – for a first look at the UI, plain HTTP will do.

1. DuckDNS

Sign in at [duckdns.org](https://www.duckdns.org), create a subdomain (`<name>`), enter the **private** address of the proxy VM as its IP. The page shows the token – it goes into the password manager; NPM needs it in a moment. The IP can also be set via the API:

```
curl "https://www.duckdns.org/update" \  
  --data-urlencode "domains=<name>" \  
  --data-urlencode "token=<duckdns-token>" \  
  --data-urlencode "ip=<ip-of-the-proxy-vm>"
```

DuckDNS answers OK. Check: `dig +short bao.<name>.duckdns.org` must return the proxy VM's IP.

2. The proxy VM

A small VM with Docker – 1 vCPU and 1 GB are enough. NPM runs as a single container; configuration and certificates live in two directories next to it:

```
# docker-compose.yml
services:
  npm:
    image: jc21/nginx-proxy-manager:latest
    restart: unless-stopped
    ports:
      - "80:80"      # HTTP, redirected to HTTPS
      - "443:443"    # HTTPS
      - "81:81"      # admin UI, LAN only
    volumes:
      - ./data:/data
      - ./letsencrypt:/etc/letsencrypt
```

```
docker compose up -d
```

The admin UI is at `http://<ip-of-the-proxy-vm>:81`. On first login (`admin@example.com / changeme`) NPM immediately demands a new address and a new password. Port 81 does not belong behind DuckDNS and not on the outside – the admin UI is the key to everything the proxy forwards.

3. The certificate

In the admin UI: *SSL Certificates* → *Add SSL Certificate* → *Let's Encrypt*.

- Domain Names: `<name>.duckdns.org` and `*.<name>.duckdns.org`
- Enable *Use a DNS Challenge*, provider **DuckDNS**
- Credentials: `dns_duckdns_token=<duckdns-token>`
- Propagation Seconds: 60 – DuckDNS takes a moment before the TXT record is visible

After a minute or two the certificate appears in the list. NPM renews it by itself as long as the token stays valid.

4. The proxy host

Hosts → *Proxy Hosts* → *Add Proxy Host*:

- Domain Names: `bao.<name>.duckdns.org`
- Scheme `http`, Forward Hostname the IP of a **worker**, Forward Port 80 – why a worker and not the control plane is in Part IV
- SSL tab: select the wildcard certificate, enable *Force SSL*

NPM passes the Host header through unchanged, and that is exactly what Traefik uses to pick the Ingress. The name therefore has to match in three places: here in the proxy host, in `server.ingress.hosts` of the values (Part I) and in `BA0_ADDR` (Part IV). Wherever this guide says `bao.example.internal`, read `bao.<name>.duckdns.org`.

Check

```
H=https://bao.<name>.duckdns.org
curl -s -o /dev/null -w '%{http_code}\n' $H/ui/
curl -s -o /dev/null -w '%{http_code}\n' $H/v1/sys/health
```

The UI answers 200 – without -k, the certificate is genuine. sys/health answers 501 before init and 503 while sealed: that is a proxy that works, and an OpenBao still waiting for Part II.

What is missing here, honestly

The DuckDNS token sits unencrypted in ./data on the proxy VM – whoever has the VM has the zone. The admin UI has a username and a password, nothing else. And NPM is not updated automatically. Acceptable for a lab; anywhere else the proxy would be the first candidate for the same hardening Part VI describes for OpenBao.

About the author

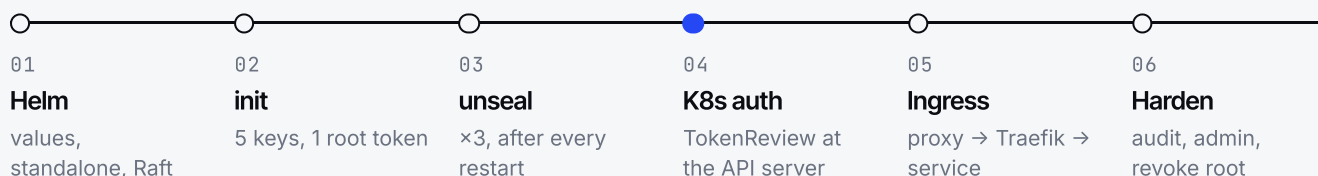
Thomas Zachmann is a freelance platform engineer based in Hamburg. He builds enterprise platforms for Kubernetes, cloud and AI workloads – from identity and secrets through CI/CD and GitOps to observability – so that the in-house team can run them without him afterwards. These Field Notes come out of that work. For project enquiries: thomaszachmann.de.

The pod is running. It is sealed. And that is exactly right.

This guide documents building an OpenBao inside a Kubernetes cluster as it actually happened: Helm values with their reasoning, init and unseal in a pod shell, Kubernetes auth with TokenReview, the path from outside through proxy and Traefik.

And the state afterwards, honestly: no audit device, the root token sitting as a file in the pod, a `service_registration` block that failed with 403 every five seconds for 22 hours. Plus the hardening that follows – in the order that rules out a lockout.

```
-- The one line that makes backups possible at all
storage "raft" {
  path = "/openbao/data"
}
```



kubernetes	RKE2 v1.36 · Traefik · Longhorn
openbao	2.6.2 · chart 0.29.4 · raft
helm	3.16 · standalone
auth	kubernetes · TokenReview
storage	10Gi PVC · Retain

37 pages · 7 parts · 3 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.
CC BY 4.0 · share freely · no warranty